PIE

PATENT INFORMATION EXTRACTION FORM

TOPIC: **Rule-Based Automated Branch Synchronization and Merge Governance System with Compliance Audit Engine**

SCHOOL: Chitkara University, Rajpura, Punjab

INVENTOR'S DETAIL WHO UPLOADED THE PATENT ON CHALKPAD

NAME: **Prof. Dr. S. B. Goyal**

EMPLOYEE CODE: **CET1003885**

MOBILE NO: **9220902139**

MAIL ID (UNIVERSITY): shyam.goyal@chitkara.edu.in

MAIL ID (PERSONAL):

OTHER INVENTORS DETAIL

NAME	EMP CODE/ROLL NO	ADDRESS WITH EMAIL AND MOBILE NO.
Simran	**2210992382**	**Chitkara University, Rajpura, Punjab, India** simran2382.be22@chitkara.edu.in 9012259242
Prof. Dr. S. B. Goyal	**CET1003885**	**Dean, Department of Computer Science & Engineering, Chitkara University, Punjab, India** shyam.goyal@chitkara.edu.in

Answer the following questions in brief: (use extra sheets if needed, submit pics/videos etc. which can help us to understand the invention better)

Q1. What was the problem?

Modern Software Development practices depend on distributed Version Control Systems to support collaboration across multiple teams and developers. Typically, repositories hold long-lasting branches, including the development branch, sprint branch and the release branch. In a good branching strategy, developers do active work in the development branch. Once the code is dev tested, it moves to the sprint branch that is more stable as compared to the dev branch and also testing will begin by QA on the stage environment, with this branch being deployed. Now, after this branch gets stabilised and QA gives a go, the code is merged in the main release branch, which is the most stable and reliable version of the system, which gets directly synced with master as per the release dates. However, in real-life industry cases, changes made in the main release branch do not always sent back to the upstream branches like the sprint and dev branches. Developers might also make the changes directly from the sprint branch and those changes will never go to the development branch, which will result in conflicts between the sprint and dev branches. Originally, what is there in sprint should

always be there in dev. These habits cause branch divergence, merge conflicts, inconsistent code states, more time spent in resolving conflicts and a greater risk of failures in future releases. As of today, in many cases, these merge conflicts or master-dev sync or master-sprint sync is mostly done by managers or team leads, which is a time and resource-consuming process. Current tools offer basic merge automation but do not have ways to enforce consistency in the lower branches, verify commit history or provide insights focused on accountability. Code changes in a primary branch are not reliably and automatically synchronized in downstream branches in the version control workflows that are currently in place. Moreover, there is no efficient method for identifying or reporting those commits that go against established branching techniques. This leads to manual efforts, frequent merge conflicts and decreased development efficiency.

Based on an empirical study, 66 projects were analysed that have approximately 81,000 merge scenarios, involving around 2 million files and over 10 million code chunks. The results derived from the study show that merge conflict resolution time is significantly influenced by factors like the number of conflicting code chunks, lines of code and the number of developers involved. Additionally, an observational study after conducting an in-situ observation reported that the conflict resolution time taken by 7 developers resolving 10 merge conflicts took around 40 to 2190 seconds, i.e. 36.5 minutes approximately, highlighting the issues that occur when different people work together on a project in a collaborative development environment.

Absence of a hierarchical automated branch strategy leads to merge conflicts, branch divergence and high manual synchronization effort in the version control systems.

Q2. How did you solve it (Inventive Step)?

The proposed solution introduces a rule-based automated branch synchronization and merge governance system at a specific time set by the user, taking into account the time zones and compliance audit engine designed to enforce downward branch consistency across the defined branch hierarchy.

Technical Solution:

1. A computer-implemented method designates the primary release branch as the authoritative source of truth for validated code from release in a multi-branch version-controlled repository.
2. The method initiates automatic downward propagation of release-validated commits from the authoritative branch to sprint and dev branches at user-defined times, including user-configured time zones and at user-defined intervals.
3. Prior to or in coordination with synchronization, the method validates the commit paths to detect unauthorised or out-of-order commits directly made in the sprint or release branches and records associated metadata containing the commit identifier, the developer identity, commit message in order to maintain daily tracking for such events.
4. The method presents the above violations recorded on a compliance audit dashboard so that each developer can prevent the violations in future and also be notified about the conflict risk conditions arising due to improper branching strategies, thereby supporting future merge conflicts reduction.
5. The steps outlined above are integrated, including automated downward synchronization enforcement, violated commits history and accountability-oriented reports that operate as one unified rule-based automated governance system, aiming to reduce merge-conflicts, lower manual synchronization efforts and preserve consistency of code in different branches.

Novelty:

The novelty of this solution arises from combining rule-based automated downward only synchronization enforcement, out-of-order commits tracking to ensure code maintenance across branches and also letting the developer know the potential error caused by out-of-order commits on the dashboard in order to reduce the merge conflicts in the future into a unified Rule-Based Automated Branch Synchronization and Merge Governance System with Compliance Audit Engine. The inventive subject matter is the integrated combination in a single platform. Existing version-control clients and continuous integration systems, considered individually, do not provide this combined governance loop as one branch-consistency and compliance solution.

Rule Engine:

A rule engine receives user configurations, including time zone, target and source branches, optional backup branch creation and also conflict resolution preference.
The engine applies:

1. Merge direction rules that permit only predefined downward synchronization (source and target branch) and forbid non-permitted direction.
2. Compliance rules that check and record out-of-order commits and path violations
3. Operational rules controlling cycle limits, backup and predefined or manual conflicts resolution.
4. The engine outputs clearly showcase the violations and success outputs logging with the developer identity.

Algorithm:

Step 1: Initialization: Load user-defined configuration, parameters specified by the user, source and target branch, schedule parameters including number of cycles, time interval, backup preference and conflict resolution method.
Step 2: On each cycle:

1) Check whether the merge direction given is allowed or not. If not, abort that merge.
2) If permitted, then check commit histories between source and target. Check out of order or violated commits and paths, if any. Append violation records using the compliance audit engine and update daily tracking.
3) If backup is opted, create a branch from the target branch to have a recovery in case of error.
4) Now merge the branches. If there is any error, then abort the merge and log the error to resolve manually.
5) Show the audit records and dashboard history.

Step 3: Report: Expose violations, synchronization results and conflict-related information through the compliance audit engine and merge governance system.

Prototype and validation:

The prototype for the above-mentioned solution is built using git-based repositories structured into release, sprint and dev branches. The implementation of the prototype is tested in a controlled development environment having an Intel CORE i7 processor and 32 GB RAM, checking most of the scenarios like out-of-order commits and downward branch sync. A total of 10-15 scenarios regarding merge conflicts and other issues. The results demonstrated successful automated downward propagation and identification of branch divergence cases.

Q3. What were the other possible solutions and why they could not be done?

The other possible solutions that we have as of today in place or solutions that couldn't be done are as follows:

1. Manual Branch Synchronization: It requires human efforts and also it's a time-consuming process to resolve conflicts created due to out-of-order commits in different branches. This leads to oversight errors and it's not that much scalable. Studies indicate that resolving merge conflicts resolution time may range from 40 seconds to over 30 minutes, depending on the code complexity, which is time costly.
2. Basic Scheduled Merge Scripts: Automates merging but lacks the logging of out-of-order commits reports and divergence intelligence.
3. Strict Access Control Policies: Prevents certain commits but does not guarantee downstream consistency.
4. Continuous Integration (CI) Pipelines: The CI Pipeline Systems ensure build stability and automated testing but do not have any branch propagation strategy in place to detect illegal commits history and code inconsistencies.

Comparative Analysis for existing solutions in the market as of today:

Solution Type	Features	Limitations
Manual Branch Synchronization	Human-controlled merging.	Resource efforts, error-prone and not scalable.
Scheduled merge scripts	Automates merging at regular intervals	Lacks audit logs, governance rules and violation detection.
Access Control Policies	Prevents unauthorized commits	Doesn't constraint correct commit flow in the downstream branches.
CI/CD Tools (Eg, Jenkins/ GitHub Actions)	Automation Pipelines and build pipelines	No enforcement for branch sync or commit violation capturing.
Git Native Tools	Rebase, Merge and Diff operations	No branch hierarchy enforcement or compliance audit engine.

All the above-mentioned solutions aim to resolve one or the other problem but not in a single platform. They provide isolated solutions but in fast-paced modern development era, the invention solved the problem with all the issues in a single platform.

Q4. What are the advantages of the solution proposed by you?

The advantages of the solution proposed above are as follows:

1. **Assure continuous downward branch synchronization:** By doing scheduled automated branch synchronization, the system will be having 100% validated flow of release-approved commits to the downstream branches, reducing merge conflicts.
2. **Improves Code Consistency from release to downstream branches:** Branch sync minimizes code inconsistencies and ensures code states across all branches.

3. **Minimize merge conflicts:** Daily sync will happen, which will reduce the merge conflict occurrences by approximately 40-50% mainly from release to downstream branches.
4. **Reduces Manual Branch Synchronization efforts:** As the process will be automated and commits will be logged, leading to 60-70% of human efforts reduction. Resulting in saving several hours of resources per week.
5. **Boost Individual Ownership for developers:** As commits will be logged, developers will feel a sense of ownership to maintain the branching rules in place.
6. **One platform providing all solutions:** As various existing software's are there which provide different solutions for different problems but not a single one provides all solutions in one place.
7. **Suitable for Agile Methodology and Enterprise Environments**
8. **Improves branch stability**

Performance Comparison (Approx. before and after this proposed solution) :

Metric	Existing Method/System	Proposed Solution
Merge Conflict Rate	High	Will be reduced by 50%
Branch Sync consistency	Unreliable	Fully enforced
Manual Efforts for Conflict Resolution	High	Will be reduced by 60% to 70%
Compliance Audit Engine	Not available directly	Complete Audit logs
Rule-Based Automation	Partial	End-to-end automated except for some error cases if caused by the merge sync

This proposed solution will resolve all the above-mentioned issues and will enhance the rule-based branch sync hierarchy and introduce accountability for the developers. These advantages collectively improve software development productivity, reduce errors and ensure reliable code across a multi-branch environment.

Q5. Explain the stepwise working of the innovation explaining all the components used in the invention and the specific function they are performing.

The proposed system automates the branch synchronization based on predefined rules and monitor commits across branches to detect branching rule violations. The system make sure that the code changes propagates from upstream branches to downstream branches, while preserving developers' work and preventing accidental code overwrite.

1. Components of the System

The proposed system consists of many logical components that work together to implement automated branch governance.
Each component is implemented as a logical processing engine that is responsible for executing specific operations based on predefined rules, which are as follows:

1.1. User Configuration Engine

This engine allows users to set all the necessary parameters needed for the correct automated branch synchronization process.

The configuration includes:

- Source Branch (Eg: master)
  The source branch is the primary branch from which code changes will be automatically down-merged to the sprint or dev.
- Target Branch (Eg: sprint)
  Target branches are the downstream branches that receive the code from upstream branches.
- Synchronization interval
  Defines the time gap between the no. of cycles user have given. Eg: Let's say user defined no of cycles as 2 and interval as 24 hrs. So, after 24 hr branch will synchronize 2nd time.
- Number of synchronization cycles
  This means no. of times user want auto sync to happen.
- Conflict resolution preference
  This determines which branch changes the user want in case of conflicts if any.
- Backup branch creation option
  This is a bool (YES/NO) whether the user wants to have a backup branch or not before auto sync.

1.2. Synchronization Schedular Engine

This engine starts the automated branch synchronization process at specified intervals.
Functions performed:

- Read the synchronization interval from the configuration provided by the user.
- Automatically conducts synchronization cycles.
- Makes sure the periodic monitoring of branch states.

1.3. Commits Analysis Engine

This engine examines commits across different branches and compares their respective commit histories

Eg: Commands such as:
"git log sprint –not dev"

This command reveals commits that are present in the sprint branch but not in the dev branch.

Functions performed:

- Discovers differences in commits between branches.
- Flags any unauthorized commits.
- Track commit metadata such as developer name, commit Id and commit msg.

1.4. Compliance Audit Engine

This engine checks whether the developers have followed the correct branching strategy for development or have done out of order commits in upstream branches.

In the original flow, commits should propagate as:

dev → sprint → master

Developers sometimes commit directly to the sprint or master branches. When these commits are detected, they will be logged by our proposed system:

- Commit ID
- Developer Name

- Commit Message
- Branch violation occurred
- Timestamp

This will help to build a dashboard for compliance monitoring to maintain accountability and traceability in the development process.

1.5. Branch Backup Engine

Before auto-sync, branch backup will be done if given by the user i.e. a branch will be created from the target branch to keep a copy of the original code and then auto-sync will happen on the target branch.

This will help the user to have a safe option to rebase if any unexpected behaviour happens in the target branch after auto sync.

Functions performed:

- Make timestamp backup branches
- Preserve original branch states before sync
- Provides a rollback option for the user

1.6. Merge Conflict Prediction and Detection Engine

Before merging branches, the system analyses differences to detect possible conflicts if any. When same lines of code is modified in both target and source branch, then merge conflicts will occur. In order to resolve this, user will be asked to accept which one as mentioned in user config module.

The system will check file differences using existing git comparison commands and detects whether conflicts can be there or not during sync.

Functions performed:

- Identify conflicting lines of code in files
- Log risky and potential conflict conditions.
- Prepare the system for resolution of those conflicts.

1.7. Rule-Based Merge Governance Engine

This engine will ensure only down-merge sync and branch hierarchy for that. This means only specific direction merge is allowed through our proposed system.

- Allowed merges:
  master → sprint
  master → dev
  sprint → dev

- Blocked merges:
  dev → sprint
  dev → master
  sprint → master

1.8. Policy-Based Conflict Resolution Engine

When conflicts are detected during sync, the system applies the rule set by user i.e. whether to accept target branch changes (Eg: sprint) or source branch changes (Eg: master).

Functions performed:

- If user has given preference to source branch, then in case of conflicts, it will keep source branch code.
- Else if user has given target branch, then that code will be merged that is present in target branch during conflict.

If conflicts can't be resolved automatically, then the system will log those commits with Ids and message and will abort the merge for manual resolution.

1.9. Logging and Audit Engine

All actions performed by the system during auto sync will be recorded in a log file.

The system will log:

- Events related to Synchronization
- Violations of Compliance
- Merge Outcome
- Identifying Conflicts
- Formation of backup branch
- Details of the developer commits

2. The Proposed System's Stepwise Operation

2.1. **System Initialization:** The system will read the configuration parameters given by the user.

2.2. **Schedular Activation:** Based on the given interval and the number of cycles by the user, the schedular initiates the sync cycle.

2.3. **Commit Analysis:** The commit analyser compares commit histories across branches and identifies diffs.

2.4. **Compliance Monitoring:** The system keeps track of unauthorized commits in higher-level branches and logs the developers' information along with the commit metadata.

2.5. **Backup Creation:** Before sync, Backup branch will be created from the target branch if opted by the user.

2.6. **Conflict Prediction:** The system will check file level changes whether any conflicts we can face during sync.

2.7. **Automated Branch Synchronization:** The merge governance engine will then perform down-merge operations between branches according to the branch parameters given by the user.

2.8. **Conflict Resolution:** If conflicts occur during sync, then the code of which branch will be kept is decided by predefined resolution option given by the user in config. Else merge will be abort and will require manual resolution.

2.9. **Logging and Reporting:** At last, logged and reported issues audit file will be created to maintain transparency.

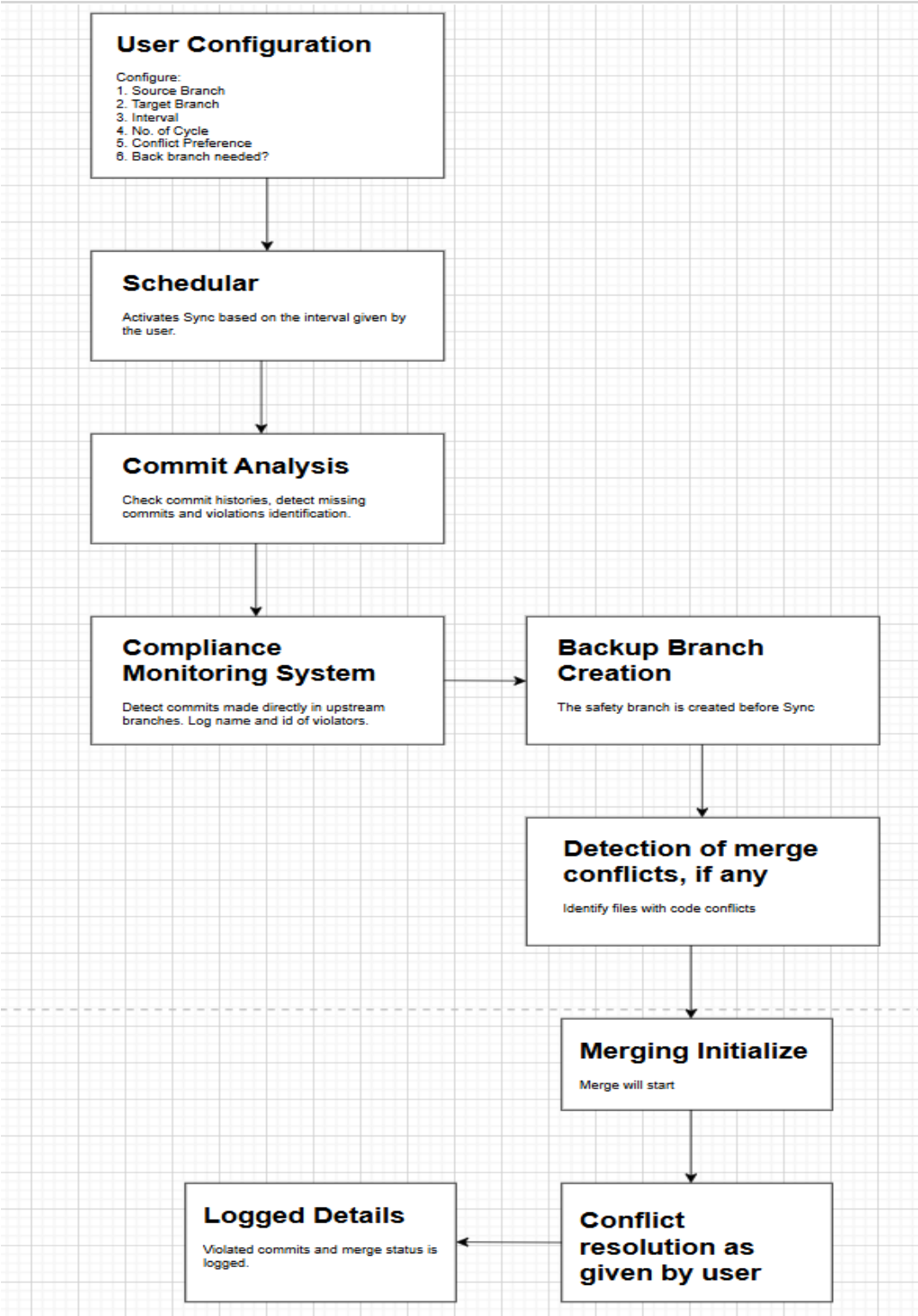
Algorithm (Pseudo-code)

```
INPUT:
   source_branch = master     //Either master or sprint as downward merge is allowed.
   target_branch = sprint     //Either sprint or dev
   no_of_cycle = 1            //How many cycles for sync
   sync_interval = 2  hrs     //After how much time each cycle should start sync again
   back_up = yes              //Either yes/no to create a backup branch or not
   conflict_resolution = master      //Which branch changes to keep in case of merge conflict if any
FOR each no_of_cycle in sync_interval:
        FOR each branch pair (source_branch -> target_branch):
                IF (source -> target) is allowed:
                        commits_diff = git_log(source_branch NOT target_branch)
                        IF commits_diff is not empty:
                                violations = git_log(target_branch NOT source_branch)
                                IF violations exist:
                                        Log violation with metadata
                                conflicts = detect_conflicts(source_branch, target_branch)
                                IF conflicts == TRUE:
                                        APPLY conflict_resolution strategy
                                                IF unresolved:
                                                        ABORT merge
                                                        LOG issue
                                                ELSE:
                                                        MERGE
                                ELSE:
                                        MERGE automatically
                                LOG Success
            ELSE:
                    BLOCK operation
                    LOG unauthorized attempt
EXIT()
```

Q6. Attach drawing hand-made/ computer made showing all the components of the invention.

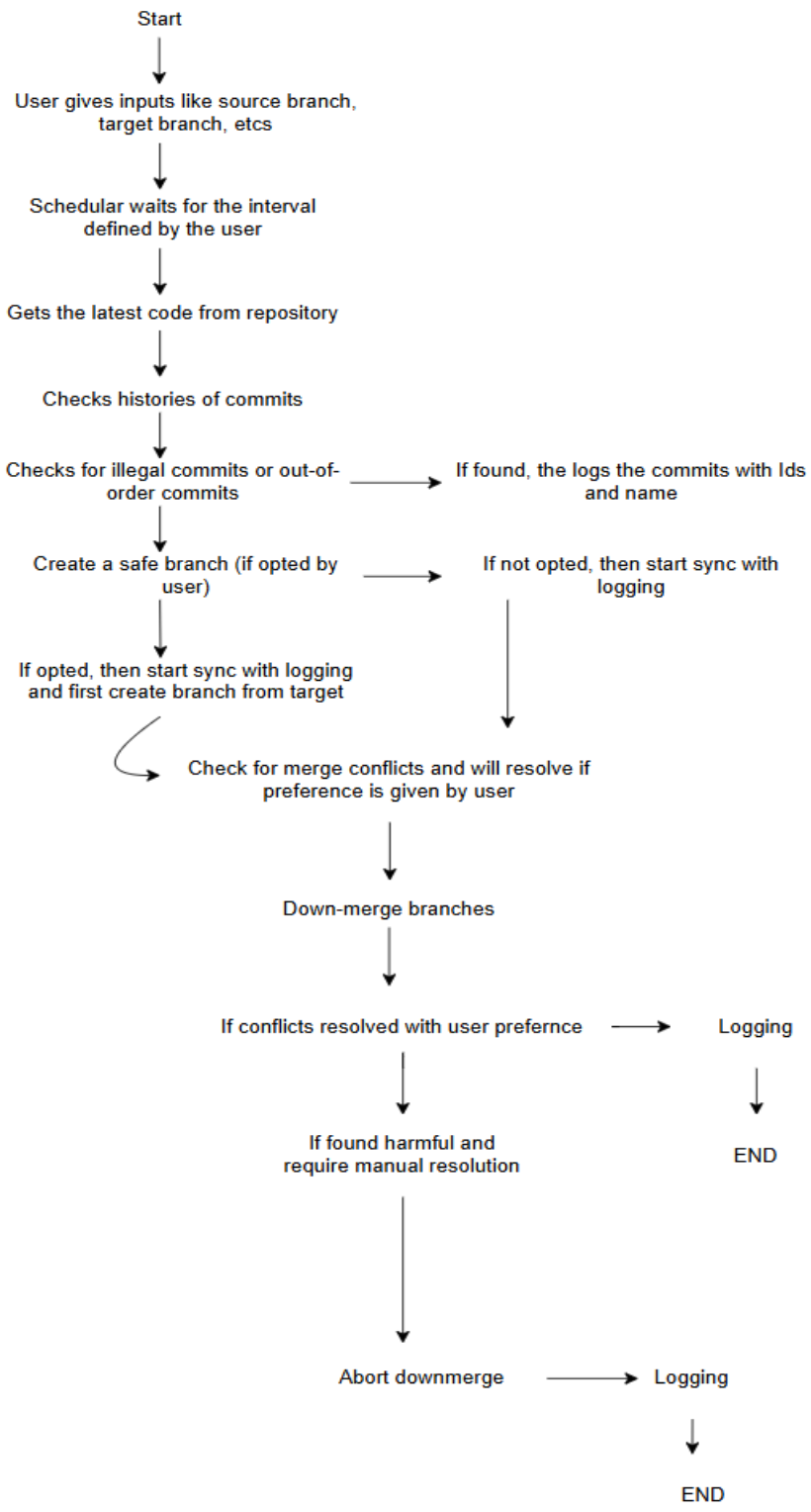
1. **System Architecture Diagram:**

   This diagram illustrates the complete system flow from user input to audit logging and successful sync engines between branches.



2. Workflow Diagram:

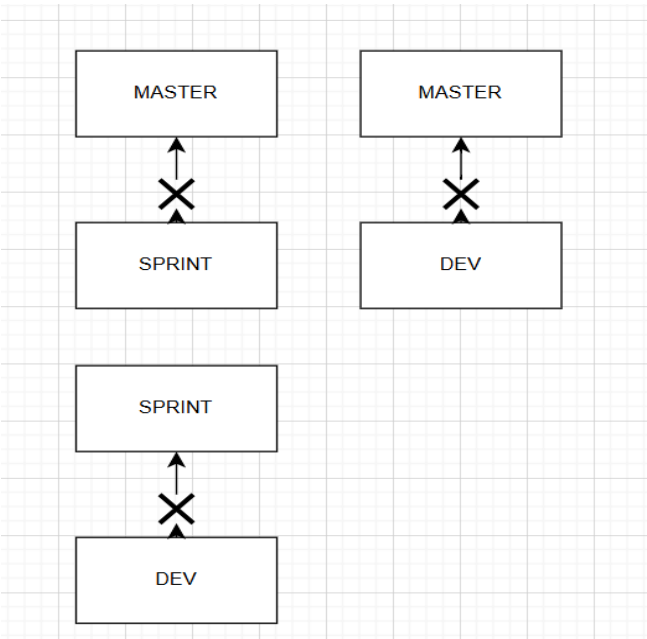
This diagram illustrates the step-by-step operational workflow through each engine, with a negate statement flow and true flow between engines.



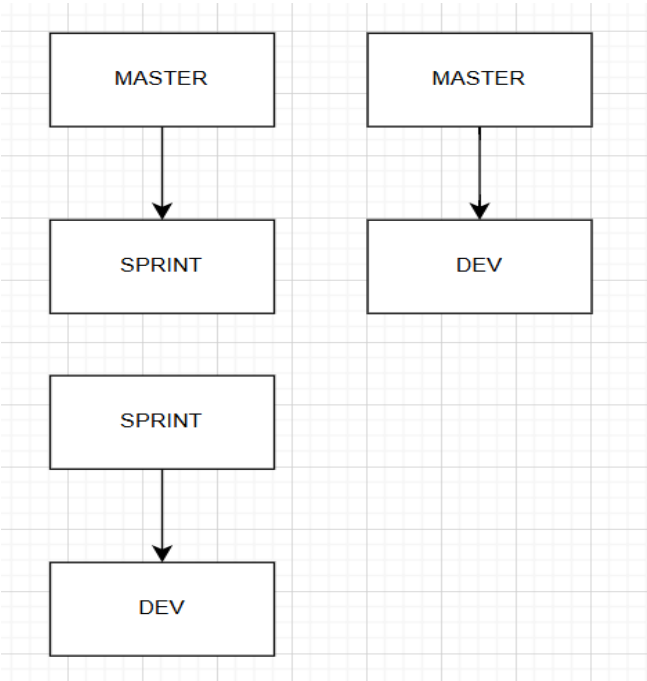
3. **Branch hierarchy:**

These two diagrams show the branch hierarchy supported for Rule-Based Automated Synchronization to happen between branches.

NOT ALLOWED

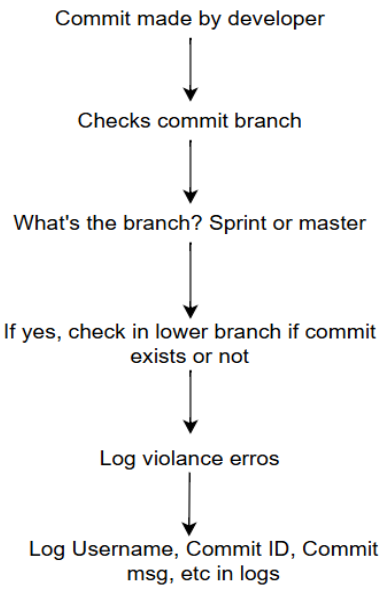


ALLOWED



4. Compliance Monitoring Flow:

This diagram and screenshot will check the developer out of order commits.




```
(venv) D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync>python demo_setup.py
Demo repository created at: ./demo_repo
  Branches: master, sprint, dev
  master commits: 5
  sprint commits: 5
  dev    commits: 6

Planted violations:
  - Direct hotfix commit on master (by Dave Senior)
  - Sprint-only feature commit (by Bob Developer)
  - Quick fix directly on sprint (by Alice Developer)
  - Conflicting utils.py changes between sprint and dev

(venv) D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync>python main.py

Automated Branch Synchronization and Compliance Monitoring System
Patent Prototype - Console Edition

Branch Synchronization & Compliance Monitor

[1]   Run Branch Sync (single pair)
[2]   Run Full Hierarchy Sync (all pairs)
[3]   View Compliance Report
[4]   Analyze Commits Across Branches
[5]   View Conflict Predictions
[6]   View Sync History
[7]   View Audit Log
[8]   Start Scheduled Sync
[9]   View / Edit Configuration
[10]  Setup Demo Repository
[0]   Exit

Patent Prototype v1.0

Select option (0): 3

--- Compliance Report ---

VIOLATIONS FOUND: 4

Sprint Branch Violations (commits in sprint not in dev)

| Commit ID | Developer | Message | Type |
|---|---|---|---|
| 6315dccb | Alice Developer | Quick fix directly on sprint branch | out-of-order: commit in sprint but not in dev |
| 2345b832 | Bob Developer | Add sprint-only feature (bypassing dev) | out-of-order: commit in sprint but not in dev |
```

5. **Merge Conflict Detection and Resolution Flow:**

This diagram checks the conflicts and asks the user which strategy to follow for merge conflicts resolution, if any. The screenshots are attached for this from the mini prototype.

```
Start Merge Process
      |
      v
Compare Files Between Branches
      |
      v
Same File Modified?
      |
 +----+----+
 |         |
Yes        No
 |         |
Detect Conflict
 |         |
Ask Merge Strategy
 |         |
Prefer Source Branch?
 |         |
 +----+----+
 |         |
Yes        No
 |         |
Keep Source   Keep Target
Changes       Changes
 |
 v
Resolve Conflict
 |
 v
Complete Merge
 |
 v
Log Merge Result
```

Screenshots merge conflict detection and resolution engine

```
Branch Synchronization & Compliance Monitor

[1]   Run Branch Sync (single pair)
[2]   Run Full Hierarchy Sync (all pairs)
[3]   View Compliance Report
[4]   Analyze Commits Across Branches
[5]   View Conflict Predictions
[6]   View Sync History
[7]   View Audit Log
[8]   Start Scheduled Sync
[9]   View / Edit Configuration
[10]  Setup Demo Repository
[0]   Exit

Patent Prototype v1.0

Select option (0): 2

--- Full Hierarchy Sync ---
    Sync Pairs

 #   Source   Target
 1   master   sprint
 2   master   dev
 3   sprint   dev

Proceed with hierarchy sync? [y/n] (y): y

Sync Result
Source: master
Target: sprint
Status: SYNCED

Commit Analysis: 0 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/sprint_20260318_000446
Conflict Prediction: 0 file(s)
Merge: success - Successfully merged master into sprint

Sync Result
Source: master
Target: dev
Status: SYNCED

Commit Analysis: 2 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/dev_20260318_000447
Conflict Prediction: 0 file(s)
```

```
Sync Result
Source: master
Target: dev
Status: SYNCED

Commit Analysis: 2 source-only, 3 target-only
Compliance: 5 violation(s)
Backup: backup/dev_20260318_000447
Conflict Prediction: 0 file(s)
Merge: success - Successfully merged master into dev

Sync Result
Source: sprint
Target: dev
Status: SYNCED_WITH_RESOLUTION

Commit Analysis: 3 source-only, 4 target-only
Compliance: 3 violation(s)
Backup: backup/dev_20260318_000448
Conflict Prediction: 1 file(s)
Merge: conflict - Merge conflicts detected in 1 file(s)
Resolution: resolved - All 1 conflict(s) resolved using source strategy
```

6. Sequence Diagram

This diagram shows the interaction between different system components during sync cycle.

```
User
 |
 | Configure Settings
 v
Scheduler
 |
 | Trigger Sync
 v
Commit Analyzer
 |
 | Compare Branches
 v
Compliance Monitor
 |
 | Detect Violations
 v
Backup Module
 |
 | Create Backup Branch
 v
Conflict Detector
 |
 | Check File Changes
 v
Merge Engine
 |
 | Perform Downmerge
 v
Conflict Resolver
 |
 | Apply Merge Strategy
 v
Logging System
 |
 | Store Logs
 v
End
```

7. Prototype Flow

This diagram represents the implementation level flow of the prototype and proposed solution.

```
Run Python Script
      |
      v
Enter Source Branch
      |
      v
Enter Target Branches
      |
      v
Enter Sync Interval
      |
      v
Enter Number of Cycles
      |
      v
Choose Conflict Strategy
      |
      v
Start Automation
      |
      v
Check Branch Differences
      |
      v
Log Compliance Violations
      |
      v
Create Backup Branch
      |
      v
Perform Downmerge
      |
      v
Log Results
      |
      v
Repeat Until Cycles Complete
```

8. Test Cases Present in the Mini Prototype (52 test cases)

```
(venv) D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync>python -m pytest tests/ -v --tb=short
============================================= test session starts =============================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync\venv\S
cripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Cousera_8th_sem\Automated_Branch_Synchronization_and_Compliance_Monitoring_System_T1\automated-branch-sync
collected 52 items

tests/test_backup.py::TestBackup::test_create_backup_branch PASSED                                  [  1%]
tests/test_backup.py::TestBackup::test_backup_preserves_state PASSED                                 [  3%]
tests/test_backup.py::TestBackup::test_list_backups PASSED                                           [  5%]
tests/test_backup.py::TestBackup::test_rollback PASSED                                               [  7%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_get_branch_commits PASSED                   [  9%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_identical_branches_no_diff PASSED           [ 11%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_diverged_branches PASSED                    [ 13%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_compare_branches PASSED                     [ 15%]
tests/test_commit_analyzer.py::TestCommitAnalyzer::test_commit_metadata PASSED                      [ 17%]
tests/test_compliance.py::TestCompliance::test_compliant_branches PASSED                            [ 19%]
tests/test_compliance.py::TestCompliance::test_sprint_violation_detected PASSED                     [ 21%]
tests/test_compliance.py::TestCompliance::test_master_violation_detected PASSED                     [ 23%]
tests/test_compliance.py::TestCompliance::test_full_compliance_report PASSED                        [ 25%]
tests/test_compliance.py::TestCompliance::test_violations_logged_to_db PASSED                       [ 26%]
tests/test_conflict_detector.py::TestConflictDetector::test_no_conflicts_identical_branches PASSED  [ 28%]
tests/test_conflict_detector.py::TestConflictDetector::test_no_conflicts_clean_divergence PASSED    [ 30%]
tests/test_conflict_detector.py::TestConflictDetector::test_conflict_detected PASSED                [ 32%]
tests/test_conflict_detector.py::TestConflictDetector::test_conflict_risk_level PASSED              [ 34%]
tests/test_conflict_resolver.py::TestConflictResolver::test_resolve_with_source_priority PASSED     [ 36%]
tests/test_conflict_resolver.py::TestConflictResolver::test_resolve_with_target_priority PASSED     [ 38%]
tests/test_conflict_resolver.py::TestConflictResolver::test_no_conflicts_returns_clean PASSED       [ 40%]
tests/test_merge_engine.py::TestMergeEngine::test_master_to_sprint_allowed PASSED                   [ 42%]
tests/test_merge_engine.py::TestMergeEngine::test_master_to_dev_allowed PASSED                      [ 44%]
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_dev_allowed PASSED                      [ 46%]
tests/test_merge_engine.py::TestMergeEngine::test_dev_to_sprint_blocked PASSED                      [ 48%]
tests/test_merge_engine.py::TestMergeEngine::test_dev_to_master_blocked PASSED                      [ 50%]
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_master_blocked PASSED                   [ 51%]
tests/test_merge_engine.py::TestMergeEngine::test_clean_merge_succeeds PASSED                       [ 53%]
tests/test_merge_engine.py::TestMergeEngine::test_merge_brings_commits PASSED                       [ 55%]
tests/test_scenarios.py::TestScenario01::test_clean_master_to_sprint PASSED                         [ 57%]
tests/test_scenarios.py::TestScenario02::test_clean_master_to_dev PASSED                            [ 59%]
tests/test_scenarios.py::TestScenario03::test_clean_sprint_to_dev PASSED                            [ 61%]
tests/test_scenarios.py::TestScenario04::test_dev_to_sprint_blocked PASSED                          [ 63%]
tests/test_scenarios.py::TestScenario05::test_dev_to_master_blocked PASSED                          [ 65%]
tests/test_scenarios.py::TestScenario06::test_sprint_to_master_blocked PASSED                       [ 67%]
tests/test_scenarios.py::TestScenario07::test_sprint_out_of_order PASSED                            [ 69%]
tests/test_scenarios.py::TestScenario08::test_master_out_of_order PASSED                            [ 71%]
tests/test_scenarios.py::TestScenario09::test_source_priority_resolution PASSED                     [ 73%]
tests/test_scenarios.py::TestScenario10::test_target_priority_resolution PASSED                     [ 75%]
tests/test_scenarios.py::TestScenario11::test_merge_abort_on_unresolvable PASSED                    [ 76%]
tests/test_scenarios.py::TestScenario12::test_backup_and_rollback PASSED                            [ 78%]
tests/test_scenarios.py::TestScenario13::test_scheduler_two_cycles PASSED                           [ 80%]
tests/test_scenarios.py::TestScenario14::test_multi_violation_report PASSED                         [ 82%]
```


```
tests/test_merge_engine.py::TestMergeEngine::test_sprint_to_master_blocked PASSED                   [ 51%]
tests/test_merge_engine.py::TestMergeEngine::test_clean_merge_succeeds PASSED                       [ 53%]
tests/test_merge_engine.py::TestMergeEngine::test_merge_brings_commits PASSED                       [ 55%]
tests/test_scenarios.py::TestScenario01::test_clean_master_to_sprint PASSED                         [ 57%]
tests/test_scenarios.py::TestScenario02::test_clean_master_to_dev PASSED                            [ 59%]
tests/test_scenarios.py::TestScenario03::test_clean_sprint_to_dev PASSED                            [ 61%]
tests/test_scenarios.py::TestScenario04::test_dev_to_sprint_blocked PASSED                          [ 63%]
tests/test_scenarios.py::TestScenario05::test_dev_to_master_blocked PASSED                          [ 65%]
tests/test_scenarios.py::TestScenario06::test_sprint_to_master_blocked PASSED                       [ 67%]
tests/test_scenarios.py::TestScenario07::test_sprint_out_of_order PASSED                            [ 69%]
tests/test_scenarios.py::TestScenario08::test_master_out_of_order PASSED                            [ 71%]
tests/test_scenarios.py::TestScenario09::test_source_priority_resolution PASSED                     [ 73%]
tests/test_scenarios.py::TestScenario10::test_target_priority_resolution PASSED                     [ 75%]
tests/test_scenarios.py::TestScenario11::test_merge_abort_on_unresolvable PASSED                    [ 76%]
tests/test_scenarios.py::TestScenario12::test_backup_and_rollback PASSED                            [ 78%]
tests/test_scenarios.py::TestScenario13::test_scheduler_two_cycles PASSED                           [ 80%]
tests/test_scenarios.py::TestScenario14::test_multi_violation_report PASSED                         [ 82%]
```


```
tests/test_scenarios.py::TestScenario15::test_full_end_to_end PASSED                                 [ 84%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_up_to_date PASSED                              [ 86%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_success PASSED                                 [ 88%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_blocked_upward PASSED                          [ 90%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_creates_backup PASSED                          [ 92%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_no_backup_when_disabled PASSED                 [ 94%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_with_conflicts_resolved PASSED                 [ 96%]
tests/test_sync_engine.py::TestSyncEngine::test_sync_logs_to_db PASSED                              [ 98%]
tests/test_sync_engine.py::TestSyncEngine::test_hierarchy_sync PASSED                               [100%]

============================================= 52 passed in 53.37s =============================================
```

References:

1. *Challenges of Resolving Merge Conflicts: A Mining and Survey Study*
   Gustavo Vale, Claus Hunsen, Eduardo Figueiredo, and Sven Apel
   Available at: https://www.se.cs.uni-saarland.de/publications/docs/VHF+22.pdf